

```
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Scanner;
import java.util.Set;

public class AlocacaoProjetos2{

    abstract static class Profissional{

        private final int idProf;
        private final String nome;
        private double valHora;

        private static final Set<Integer> idsDisp = new HashSet<>();
        //static Random geracao = new Random();

        public Profissional (int idProf, String nome, double valHora){
            this.idProf = idProf;
            idsDisp.add(idProf);
            this.nome=nome;
            this.valHora=valHora;
        }

        /*private static int GenUnicId(){
            int novoId;
            if(idsDisp.size()>=10){
                throw new RuntimeException("Numero maximo de
funcionarios atingidos");
            }
            do{
                novoId=geracao.nextInt(100);
            }while(idsDisp.contains(novoId));
            idsDisp.add(novoId);
            return novoId;
        }*/

        public static int getQtndCadastrado(){
            return idsDisp.size();
        }

        public static boolean IdExiste(int id){
            return idsDisp.contains(id);
        }
    }
}
```

```

    }

    public abstract double calcCustoMensal(int valHora);
    public void setValHora(double
novoSalario){this.valHora=novoSalario;}

    public int getIdProf() {return idProf;}
    public String getNome() {return nome;}
    public double getValHora() {return valHora;}
}

interface promovivel{
    void promocao(double valAumento);
}

static class Dev extends Profissional{
    public Dev(int idProj, String nome, double valHora){
        super(idProj, nome, valHora);}
    @Override
    public double calcCustoMensal(int valHora){
        return this.getValHora()*8;
    }/*
    @Override
    public void promocao(double valAumento){
        double novoSalario = this.getValHora()*(1+valAumento/100);
        this.setValHora(novoSalario);
        System.out.println("Salario atualizado para:
"+novoSalario);
    }*/
}

static class Gerente extends Profissional implements promovivel{
    public Gerente(int idProj, String nome, double valHora){
        super(idProj, nome, valHora);}
    @Override
    public double calcCustoMensal(int valHora){
        return (this.getValHora()*8)+1000;
    }
    @Override
    public void promocao(double valAumento){
        double novoSalario = this.getValHora()*(1+valAumento/100);
        this.setValHora(novoSalario);
    }
}

```

```

        System.out.println("Salario atualizado para:
"+novoSalario);
    }
}

static class Projeto{
    private final int idProj;
    private final String nomeProj;
    private final Gerente gerenteResp;
    private final List<Dev> equipeDevs;

    private static final Set<Integer> idsDisp = new HashSet<>();
    //static Random geracao = new Random();

    /*private static int GenProjId(){
        int novoId;
        if(idsDisp.size()>=10){
            throw new RuntimeException("Numero maximo de
funcionarios atingidos");
        }
        do{
            novoId=geracao.nextInt(100);
        }while(idsDisp.contains(novoId));
        idsDisp.add(novoId);
        return novoId;
    }*/

    public Projeto(int idProj, String nomeProjeto, Gerente
gerente){
        this.idProj=idProj;
        idsDisp.add(idProj);
        this.nomeProj=nomeProjeto;
        this.gerenteResp=gerente;
        this.equipeDevs= new ArrayList<>();
    }

    public void addDev(Dev dev){
        this.equipeDevs.add(dev);
    }

    public static boolean IdExiste(int id){
        return idsDisp.contains(id);
    }
}

```

```

        public double calcularCustoTotalProjeto(int horasMes) {
            double total = gerenteResp.calcCustoMensal(horasMes);

            for (Dev d : equipeDevs) {
                total += d.calcCustoMensal(horasMes);
            }

            return total;
        }

        public int getIdProj() {return idProj;}
        public String getNomeProj() {return nomeProj;}
        public Gerente getGerenteResp() {return gerenteResp;}
        public List<Dev> getEquipeDevs() {return equipeDevs;}
    }

    public static void main(String[] args) {
        System.out.println("Sistema iniciando");
        for(int i=0;i<3;i++){
            System.out.println(".");
            System.out.println("..");
            System.out.println("...");
            try { Thread.sleep(400); } catch (InterruptedException e)
        {}

    }

    List<Profissional> ListaFuncionarios = new ArrayList<>();
    List<Projeto> ListaProjetos = new ArrayList<>();
    int opcao;
    try (Scanner entrada = new Scanner(System.in)) {
        do{
            System.out.println("Sistema de alocação de projeto");
            System.out.println("1. Cadastrar Profissional
(Dev/Gerente)");
            System.out.println("2. Criar Projeto e Associar
Responsável");
            System.out.println("3. Aplicar Promoção/Aumento");
            System.out.println("4. Relatório de Custos");
            System.out.println("0. Sair");
            System.out.print("Escolha uma opção: ");

            opcao = entrada.nextInt();
            entrada.nextLine();
            switch(opcao){

```

```

        case 1 -> {
            int cadastrados =
Profissional.getQtndCadastrado();
            int limite = 10;

            if (cadastrados >= limite) {
                System.out.println("\n[ERRO] Limite de 10
funcionários atingido. Remova alguém ou reinicie o sistema.");
            } else {
                int espacoDisponivel = limite - cadastrados;
                System.out.println("\nEspaço disponível para
novos cadastros: " + espacoDisponivel);
                System.out.print("Quantos funcionários deseja
adicionar agora? ");

                int qtndFunc = entrada.nextInt();
                entrada.nextLine();

                if (qtndFunc > espacoDisponivel || qtndFunc <=
0) {
                    System.out.println("[ERRO] Quantidade
inválida. Você só pode cadastrar mais " + espacoDisponivel + ".");
                } else {
                    for (int i = 0; i < qtndFunc; i++) {
                        System.out.println("\n--- Cadastro do
Funcionário " + (i + 1) + " ---");
                        int tipofunc;
                        do {
                            System.out.print("Tipo: Dev [1] ou
Gerente [2]: ");

                            tipofunc = entrada.nextInt();
                            entrada.nextLine();
                            if (tipofunc != 1 && tipofunc != 2)
System.out.println("Opção inválida!");
                        } while (tipofunc != 1 && tipofunc !=
2);

                        int idFunci;
                        do {
                            System.out.print("Digite o ID
desejado: ");

                            idFunci = entrada.nextInt();
                            entrada.nextLine();

```

```

        if (Profissional.IdExiste(idFunci))
        {
            System.out.println("Este ID já
está em uso! Tente outro.");
        }
    } while
(Profissional.IdExiste(idFunci));

    System.out.print("Nome: ");
    String nomeFunci = entrada.nextLine();

    System.out.print("Valor/Hora: ");
    double salHora = entrada.nextDouble();
    entrada.nextLine();

    if (tipofunc == 1) {
        ListaFuncionarios.add(new
Dev(idFunci, nomeFunci, salHora));
        System.out.println("Dev cadastrado
com sucesso!");
    } else {
        ListaFuncionarios.add(new
Gerente(idFunci, nomeFunci, salHora));
        System.out.println("Gerente
cadastrado com sucesso!");
    }
}
}
}

case 2->{
    System.out.println("Deseja cadastrar quantos
projetos: ");

    int qtndProj=entrada.nextInt();
    entrada.nextLine();
    for(int i=0;i<qtndProj;i++){

        System.out.println("ID do Gerente
Responsável:");

        int idG = entrada.nextInt();
        entrada.nextLine();
        Gerente gSelecionado = null;

```

```

        for (Profissional p : ListaFuncionarios) {
            if (p.getIdProf() == idG && p
instanceof Gerente) {

                gSelecionado = (Gerente) p;
                break;
            }
        }

        if (gSelecionado == null) {
            System.out.println("Erro: Gerente não
encontrado ou ID pertence a um Dev.");
        } else {
            System.out.println("ID do Projeto:");
            int idProj = entrada.nextInt();
            entrada.nextLine();
            System.out.println("Nome do Projeto:");
            String nomeP = entrada.nextLine();

            Projeto novoProj = new Projeto(idProj,
nomeP, gSelecionado);

            String continuar;
            do {
                System.out.println("Deseja
adicionar um Desenvolvedor a este projeto? [s/n]");
                continuar = entrada.next();

                if
(continuar.equalsIgnoreCase("s")) {
                    System.out.println("Digite o ID
do Dev:");

                    int idD = entrada.nextInt();
                    Dev dSelecionado = null;

                    for (Profissional p :
ListaFuncionarios) {
                        if (p.getIdProf() == idD &&
p instanceof Dev) {
                            dSelecionado = (Dev) p;
                            break;
                        }
                    }
                }
            } while (continuar.equalsIgnoreCase("n"));
        }
    }
}

```

```

        if(dSelecioneado != null) {
novoProj.addDev(dSelecioneado);
        System.out.println("Dev " +
dSelecioneado.getNome() + " adicionado!");
        } else {
        System.out.println("ID inválido
ou profissional não é um Dev.");
        }
    }
    } while
(continuar.equalsIgnoreCase("s"));

        ListaProjetos.add(novoProj);
    }
}

case 3->{
    System.out.println("Deseja fazer o aumento de
salario de algum funcionario, SIM[y], NAO[n]");
    if(!entrada.next().equalsIgnoreCase("y")){
        System.out.println("Operacao encerrada");
    }
    else{
        System.out.println("=== Funcionarios
Cadastrados ===");
        for(Profissional prof : ListaFuncionarios){
            System.out.println("ID:
"+prof.getIdProf()+" | Nome: "+prof.getNome());
        }
        System.out.println("Digite o ID do
funcionario escolhido:");
        int funcEscolhido = entrada.nextInt();
        entrada.nextLine();
        for(Profissional prof : ListaFuncionarios){
            if(prof.getIdProf()==funcEscolhido){
                if(prof instanceof promovivel){
                    System.out.println("Funcionario
elegivel, por favor digite a porcentagem do aumento: ");
                    double porcentagem =
entrada.nextDouble();

```



```

                                entrada.nextLine();

                                promovivel p = (promovivel)
prof;

                                p.promocao(porcentagem);
                                }
                                else{
                                    System.out.println("Cargo sem
permissao de aumento");
                                }
                                break;
                                }
                            }
                        }
                    }
                case 4->{

if(ListaProjetos.isEmpty()){System.out.println("Nenhum projeto
cadastrado");}

                    else{
                        System.out.println("Informe a carga horaria
mensal: ");

                        int horaMes = entrada.nextInt();
                        entrada.nextLine();
                        for (Projeto proj : ListaProjetos) {
                            System.out.println("PROJETO: " +
proj.getNomeProj());

                            System.out.println("LIDERANÇA: " +
proj.getGerenteResp().getNome());

                            System.out.print("EQUIPE: ");
                            if (proj.getEquipeDevs().isEmpty()) {
                                System.out.print("Nenhum dev
alocado.");

                                } else {
                                    for (Dev d : proj.getEquipeDevs())
{

                                        System.out.print(d.getNome() + " |

");
                                    }
                                }
                            }
                        }
                    }
                }
            }
        }
    }
}

```

```

        System.out.println("\nCUSTO TOTAL: R$ " +
proj.calcularCustoTotalProjeto(horaMes));

System.out.println("-----");
    }
}
}
case 0->{System.out.println("Saindo...");}
default->{System.out.println("Opcao invalida, tente
numeros de 0 a 4");}
}
}while(opcao!=0);
}
}
}
}

```

Com certeza! Esse seu código virou uma verdadeira "aula" de Programação Orientada a Objetos (POO). Ele utiliza desde a estrutura básica de repetição até conceitos avançados de arquitetura de software.

Aqui está a explicação detalhada do que cada peça faz:

1. Pilares da Orientação a Objetos (POO)

Abstração (abstract static class Profissional)

Você definiu que `Profissional` é uma classe abstrata. Isso significa que você não pode dar um "new Profissional". Ela serve apenas como um **molde**. No mundo real, ninguém é apenas um "profissional", a pessoa é um "Dev" ou um "Gerente".

Herança (class Dev extends Profissional)

O `extends` faz com que `Dev` e `Gerente` herdem tudo o que o `Profissional` tem (ID, Nome, Valor/Hora). Isso evita repetição de código (o famoso princípio DRY - *Don't Repeat Yourself*).

Polimorfismo (@Override e instanceof)

- **@Override**: O método `calcCustoMensal` existe no pai, mas se comporta de um jeito no `Dev` e de outro no `Gerente` (que tem bônus). O Java decide na hora qual regra usar.
- **instanceof**: Usado no seu `main` para checar: "Esse profissional que achei é um Gerente?". Isso permite tratar objetos diferentes de formas específicas.

Encapsulamento (private, getters e setters)

Ao usar `private`, você protege os dados. Ninguém de fora pode mudar o `idProf` ou o `nome` diretamente. A única forma de interagir com esses dados é através dos métodos que você permitiu (Getters e Setters).

2. Estruturas de Dados e Coleções

List e ArrayList

Diferente de um Array comum (que tem tamanho fixo), a `ArrayList` é dinâmica. Você pode adicionar quantos itens quiser (até o limite da memória). Você usou uma para os funcionários e outra para os projetos.

Set e HashSet (idsDisp)

O `Set` é uma coleção que **não permite elementos duplicados**. Você o usou com maestria para garantir que nenhum ID de funcionário ou de projeto fosse repetido.

3. Comandos e Estruturas de Controle

switch com "Arrow Syntax" (case 1 -> { ... })

É a versão moderna do Java (14+). Ela é mais limpa e elimina a necessidade do comando `break` tradicional, pois o bloco `{ }` já delimita o escopo de cada opção.

do-while vs while

- **do-while**: Executa o código **pelo menos uma vez** antes de testar a condição (ideal para menus).
- **while**: Testa a condição antes de começar (usado para validar IDs).

static (Métodos e Atributos)

Quando você coloca `static` em `idsDisp` ou no método `IdExiste`, você diz que aquilo pertence à **Classe** e não ao **Objeto**. Todos os funcionários compartilham o mesmo "banco de dados" de IDs.

4. Relações entre Objetos

Agregação (O Projeto e sua Equipe)

Dentro da classe `Projeto`, você tem:

- `private Gerente gerenteResp;` (Relação 1 para 1)
- `private List<Dev> equipeDevs;` (Relação 1 para N)

Isso simula o mundo real, onde um projeto possui um líder e vários membros na equipe. Quando você chama `calcularCustoTotalProjeto`, o projeto "percorre" essa lista e soma o custo de cada um.

5. Entrada e Saída (I/O)

Scanner e o "Buffer" (`entrada.nextLine()`)

O comando `nextInt()` lê apenas o número, mas deixa o "Enter" (o caractere `\n`) sobrando no teclado. O seu `entrada.nextLine()` serve para "limpar" esse lixo, evitando que o programa pule a pergunta do nome do próximo funcionário.

`try (Scanner entrada = ...)`

Isso se chama *Try-with-resources*. Ele garante que o `Scanner` seja fechado automaticamente quando o programa terminar, evitando vazamento de recursos no computador.

Resumo do Fluxo do seu Código:

1. **Entrada:** Usuário interage com o Menu.
2. **Processamento:** O código valida regras de negócio (limite de 10, IDs únicos, tipos de cargo).
3. **Armazenamento:** Objetos são criados e guardados em Listas.
4. **Saída:** O sistema percorre as listas e gera o relatório financeiro usando Polimorfismo.

Você gostaria de fazer um teste rodando o código agora ou quer que eu revise a formatação de alguma parte específica antes de você finalizar?